9 Sorting: Selection Sort

AGES 8-12

MASCOT GOES HERE

KID INSTRUCTIONS

Last activity, you bubble-sorted by comparing pairs and swapping. But that's not the only way! What if instead, you just FOUND the smallest number and put it first? Meet selection sort — a totally different way to do the same job.

Selection

Selection — To pick or choose something. In selection sort, you SELECT the smallest item each round and put it in its place.
Example: When you pick the BEST candy from a bowl, you're selecting — just like the algorithm picks the smallest number!

How It Works – Selection Sort

Selection sort is the opposite of bubble sort's "compare and swap" approach. Instead of comparing pairs, you scan for the smallest and place it.

1. Look at the whole list and FIND the smallest item
2. SWAP it with the item at the FIRST position
3. That position is now DONE — leave it alone!
4. Repeat with the rest of the list (ignore the sorted part)
5. Stop when only one item is left — it's sorted!

Worked Example – Sort: [4] [2] [7] [1]

Start: [4] [2] [7] [1]

Sel 1: Smallest is 1. Swap with 4. → [1] [2] [7] [4]

Sel 2: Smallest is 2. No swap needed. → [1] [2] [7] [4]

Sel 3: Smallest is 4. Swap with 7. Done! → [1] [2] [4] [7]

Notice — each round, ONE item gets locked into its final place. That's the difference: bubble sort moves items little by little across many passes, but selection sort puts each item where it belongs in ONE move.

Basic Version

Pick and Place — Use selection sort to put these numbers in order from smallest to largest. After each selection, write the new list.

Start: [6] [3] [9] [1]

After Selection 1: [ ] [ ] [ ] [ ]

After Selection 2: [ ] [ ] [ ] [ ]

After Selection 3: [ ] [ ] [ ] [ ]

Which item got locked in FIRST?

____

How many selections did it take?

____

Challenge Version

PART A – Sort the SAME List Two Ways

Use both algorithms on this list. Then compare!

List: [8] [3] [5] [1] [4]

BUBBLE SORT

Start: [8] [3] [5] [1] [4]

Pass 1: [ ] [ ] [ ] [ ] [ ]

Pass 2: [ ] [ ] [ ] [ ] [ ]

Pass 3: [ ] [ ] [ ] [ ] [ ]

SELECTION SORT

Start: [8] [3] [5] [1] [4]

After Sel. 1: [ ] [ ] [ ] [ ] [ ]

After Sel. 2: [ ] [ ] [ ] [ ] [ ]

After Sel. 3: [ ] [ ] [ ] [ ] [ ]

After Sel. 4: [ ] [ ] [ ] [ ] [ ]

Did both algorithms get the same final answer?

○ YES ○ NO

PART B – Compare the Two Algorithms

Q1: Which algorithm felt easier to DO by hand? Why?

→ ______________________________

→ ______________________________

Q2: If you had a list of 100 items, which would you choose — bubble sort or selection sort? Why?

→ ______________________________

→ ______________________________

Self-Check

- ☐ I followed the selection sort worked example and saw items "lock in"
- ☐ I sorted [6, 3, 9, 1] using selection sort
- ☐ I sorted the SAME list with both algorithms and compared them
- ☐ I can explain how selection sort is different from bubble sort

Reflection Box

Sorting is one tiny problem, and we already have TWO different algorithms for it. Why might programmers want more than one way to solve the same problem?

Bonus: Algorithm Race!

Time for a race! Grab 8 small items (cards, coins, candies — whatever you have). Mix them up. Then ask a friend or family member to race you: ONE person uses bubble sort, the OTHER uses selection sort. Who finishes first?

Race Results:

My algorithm: ○ Bubble Sort ○ Selection Sort

Their algorithm: ○ Bubble Sort ○ Selection Sort

Who won?

How long did it take you? *(rough estimate — seconds, minutes?)*

Was the WINNER's algorithm really faster? Or did they just sort faster as a person? What's the difference?

TEACHER / PARENT NOTE

This activity introduces one of the deepest insights in computer science: the same problem can be solved by different algorithms, with different trade-offs. Kids see that selection sort and bubble sort produce identical results but feel different to execute; in real code, the algorithm choice affects speed, memory use, and readability. Discussion prompt: Ask your child to think of a real-life task (making a sandwich, cleaning a room, getting dressed) and see if they can come up with TWO different "algorithms" to do it.